

Pulse Programming Language

Language User Guide

A practical reference for writing, running, and debugging Pulse programs

Final Year Project

Daud Anjum

2026-09-15

1. Introduction

Pulse is an AI-native programming language with Python-like syntax, designed to make AI/ML prototyping fast and accessible without requiring external library installation. This guide covers the core language, the standard library, the command-line interface, error handling, and Pulse's Python interoperability feature (pybridge).

Note: Core language syntax and standard library module names in this guide are drawn from the interpreter's own built-in reference (pulse.py --info) and from source review of the interpreter (src/parser.py, src/tokens.py, src/stdlib/io_module.py). Behavior marked "Confirmed" was directly observed running against the live interpreter during this project; behavior not marked as such reflects the language's own documentation and has not been independently re-verified line by line in this guide.

2. Getting Started

2.1 Running a Program

```
python pulse.py path/to/program.pul
```

2.2 Interactive Mode (REPL)

```
python pulse.py
```

Starts an interactive session. Variables, functions, and classes defined on one line remain available on subsequent lines for the duration of the session. Type exit, exit(), quit, or quit() to leave, or press Ctrl+Z / Ctrl+D.

2.3 Command-Line Options

Flag	Effect
(no flag)	Run the given .pul file, or start the REPL if no file is given.
--time	Print a breakdown of lexing/parsing/resolving/interpreting time after running a file.
--info	Print the language's built-in syntax and standard library reference.
--log	Log any error to <file>.log as JSON Lines, next to the source file (file mode only; not active in REPL).

```
python pulse.py program.pul --time
python pulse.py program.pul --log
python pulse.py --info
```

3. Language Basics

3.1 Variables and Types

```
x = 10
name = "Pulse"
is_ready = True
nothing = null
x += 1
x -= 1
```

Type	Examples
number	42, 3.14, -7, 0.5
string	"hello", 'world'
boolean	True, False
null	null (or None)
list	[1, 2, 3]
dict	{ "key": "value" }
range	range(0, 10)
tensor	@[[1, 2], [3, 4]]

3.2 Operators

Category	Operators
Arithmetic	+ - * / // % **
Comparison	== != < > <= >= is is not
Logical	and or not
Membership	in not in
Chained comparison	0 < x < 10
Pipe	value > function
Ternary	x if condition else y

3.3 Strings and f-strings

```
greeting = "Hello, " + name
message = f"Hello, {name}!"
block = """multi
line"""
```

3.4 Control Flow

```
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

while x > 0:
    x -= 1

for item in [1, 2, 3]:
    print(item)

for i, val in enumerate(items):
    print(i, val)
```

match / case

```
match status:
    case 200:
        print("OK")
    case 404 | 405:
        print("Not Found")
    case n if n >= 500:
        print("Server Error")
    case _:
        print("Unknown")
```

3.5 Functions

```
def add(a, b):
    return a + b

def greet(name, msg="hello"):
    print(f"{msg}, {name}!")

def sum_all(*args):
    total = 0
    for n in args:
        total += n
    return total

double = lambda x: x * 2
```

Note: Pulse has no tuple literals and no multi-value return via comma syntax (return a, b is invalid). Return a list instead and index into it: return [a, b].

3.6 Classes

```
class Animal:
    name = "unknown"
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f"{self.name} speaks")

class Dog(Animal):
    def speak(self):
        print(f"{self.name} barks")

class Util:
    static def helper():
        print("no self needed")
```

Note: Pulse uses the static keyword before def for static methods, rather than Python's @staticmethod decorator.

3.7 Error Handling

```
try:
    result = divide(a, b)
except ValueError as e:
    print("Value error")
except:
    print("Unknown error")
else:
    print("Success")
finally:
```

```

    print("Done")

raise ValueError("something went wrong")

```

3.8 Lists and List Operations

```

nums = [3, 1, 4, 1, 5]
nums.append(9)
nums.pop()
nums.insert(0, 0)
nums.sort()
nums.sort(key=lambda x: -x, reverse=True)
nums.reverse()
nums.extend([6, 7])
nums.map(lambda x: x * 2)
nums.filter(lambda x: x > 2)
nums.index(4)
nums.find(99)          # returns -1 if not found
nums.count(1)
nums.contains(3)
nums.slice(1, 3)
nums.clear()

```

List Comprehensions

```

squares = [x * x for x in [1, 2, 3, 4]]
positives = [x for x in nums if x > 0]

```

3.9 Strings, Dicts

```

s.upper()    s.lower()    s.trim()
s.split()    s.join(lst)  s.replace(old, new)
s.starts_with(p)    s.ends_with(p)
s.contains(sub)      s.length()
s.find(sub)  s.index(sub) s.count(sub)
s.format(a, b)

d.keys()    d.values()    d.items()
d.has(key)  d.remove(key)  d.length()

```

3.10 Imports and Unpacking

```

import math
from math import sqrt, pi
import math as m

a, b = pair
del x
del nums[0]
del d["key"]

```

4. Tensors

Tensors are a first-class, built-in type in Pulse -- no import required for the type itself, though numeric ML operations live in the models/preprocess modules.

```

t = @[[1, 2, 3], [4, 5, 6]]
t.shape    t.ndim    t.size    t.dtype
t.T        t.flatten() t.reshape(2, 3)
t.sum()    t.mean()    t.min()    t.max()

```

```

t + s      t - s      t * s      t / s
t @ s      # matrix multiply
t[0]       t[0, 1]     t[1:]

```

Note: The @ sigil only works as literal tensor syntax (@[[...]]) or as the matmul operator between two existing tensors -- it cannot be used as a prefix to convert an arbitrary list variable into a tensor (e.g. @my_list is invalid). To convert a runtime list into a tensor, use tensor(my_list).

5. Standard Library Reference

5.1 math

```

sqrt, floor, ceil, log, log2, log10, exp,
sin, cos, tan, abs, pow, pi, e, inf, tau

```

5.2 io

Function	Description
read_file(path)	Read a file's full contents as a string. (Confirmed)
write_file(path, content)	Overwrite a file with the given string content. (Confirmed)
append_file(path, content)	Append a string to the end of a file. (Confirmed)
file_exists(path)	Return True if a file exists at path. (Confirmed)
read_lines(path)	Return a file's lines as a list of strings, newline-stripped. (Confirmed)

5.3 os, time, random

```

os      getcwd, chdir, listdir, mkdir, makedirs, rmdir, removedirs, rmtree,
        remove, rename, copy, exists, is_file, is_dir, is_abs, join, basename,
        dirname, abspath, splitext, split, getsize, stat, getenv, setenv,
        env_vars, platform, sep
time    now, clock, sleep
random  random, randint, uniform, randrange, choice, choices, sample,
        shuffle, gauss, normalvariate, expovariate, triangular, seed, get_state

```

5.4 AI/ML Modules: models, preprocess, metrics, datasets, learn

These modules are the core of Pulse's AI-native design and were directly exercised by this project's sample programs.

Module	Confirmed usage
datasets	datasets.iris(), datasets.diabetes() return a dataset object. Access fields with dataset.labels(), dataset.features() (attribute access, not dataset["data"]). (Confirmed)
preprocess	preprocess.train_test_split(X, y, test_size=0.2) returns X_train, X_test, y_train, y_test. (Confirmed)
models	models.LinearRegression(), models.LogisticRegression(), models.KMeans(k=n). model.train(X, y, auto_preprocess=True, verbose=True); model.predict(X); model.explain(feature_names). predict() requires tensor input -- convert lists first with tensor(list).

Module	Confirmed usage
metrics	metrics.r2(y_true, y_pred), metrics.rmse(...), metrics.mae(...), metrics.accuracy(...), metrics.classification_report(...). classification_report() prints its own formatted table as a side effect in addition to returning a value -- avoid printing the return value again. (Confirmed)
learn	learn.example(name), learn.topics -- worked examples for linear_regression, logistic_regression, knn, decision_tree, random_forest, kmeans, neural_network. (Documented, not independently re-verified in this project)

Worked Example: Classification

```
import datasets
import models
import preprocess
import metrics

data = datasets.iris()
X = data.features()
y = data.labels()

X_train, X_test, y_train, y_test = preprocess.train_test_split(X, y, test_size=0.25)

model = models.LogisticRegression()
model.train(X_train, y_train, auto_preprocess=True, verbose=True)

predictions = model.predict(X_test)
accuracy = metrics.accuracy(y_test, predictions)
print(f"Accuracy: {accuracy}")
```

6. Python Interoperability (pybridge)

The pybridge module lets Pulse code call installed Python packages or a user's own Python files directly, with automatic value conversion at the boundary (numbers, strings, booleans, lists, dicts, and numpy arrays convert automatically both ways). (Confirmed)

```
import pybridge

stats = pybridge.import_module("statistics")
print(stats.mean([1, 2, 3, 4]))

helpers = pybridge.load_file("helpers.py")
print(helpers.greet("Pulse"))
```

Note: pybridge executes arbitrary Python code with the full privileges of the Pulse interpreter process. It is intended for trusted, developer-authored code (e.g. a project's own helper functions, or well-known installed packages) -- do not use it to run untrusted Python source.

7. Error Handling and Debugging

Pulse reports errors with the failing stage, the message, the file/line/column, a source snippet, and a caret pointing at the failure location. Error stages include Lexical Error, Syntax Error, Semantic Error, and Runtime Error (the latter includes a call-stack traceback when the error occurred inside function calls).

7.1 Logging Errors to a File

Pass `--log` when running a file to additionally write a JSON Lines entry for any error to `<file>.log`, alongside the normal terminal output. (Confirmed)

```
python pulse.py program.pul --log
{"timestamp": "...", "stage": "Syntax Error", "type": "PulseSyntaxError",
"message": "Expected ')' after arguments", "filename": "<pulse>",
"line": 2, "column": 30, "context": "...", "hint": null}
```

Note: `--log` is active in file mode only; it has no effect in the REPL.

8. Known Language Differences from Python

The following were identified during this project's development and are worth knowing before porting Python code to Pulse:

- No tuple literals -- (a, b) only groups a single expression; use lists [a, b] instead.
- No multi-value return via comma syntax -- return a, b is invalid; return a list and index into it.
- `static` replaces Python's `@staticmethod` decorator.
- Dataset-like objects (e.g. from `datasets.iris()`) use method access (`data.features()`), not dict-style indexing (`data["data"]`).
- The `@` sigil is reserved for tensor literals and the `matmul` operator; it cannot prefix an arbitrary variable to convert it to a tensor -- use `tensor()` for that.

9. Where to Go Next

- Run `python pulse.py --info` at any time for the interpreter's own built-in quick reference.
- See `examples/ai/` for working regression, classification, clustering, and chatbot programs.
- See `examples/bridge/` for a working pybridge demonstration.
- See the project's Testing and Validation Report and Comparison & Evaluation Study for verified behavior, known limitations, and benchmark data.